

DAY 11: NUMPY FUNDAMENTALS

60-Day Data Science Learning Journey | Muhammad Ahmad

Goal: Move from core Python into numerical computing and Data Science with NumPy.

Learning Path

NumPy → Arrays → Dimensions → Shape/Size/Dtype → Array Creation → Indexing/Slicing → Arithmetic → Statistics → Reshape → Random Data → Boolean Filtering → Student Marks Mini Project

1. What is NumPy?

NumPy means Numerical Python. It is a Python library for numerical computing and multidimensional arrays. It is a foundation for tools such as Pandas, Matplotlib and scikit-learn.

```
import numpy as np
```

2. Installation and Version

```
pip install numpy

import numpy as np
print(np.__version__)
```

3. First NumPy Array

```
import numpy as np

numbers = np.array([10, 20, 30, 40, 50])
print(numbers)
print(type(numbers))
```

4. Python List vs NumPy Array

```
marks_list = [80, 75, 90, 85]
marks_array = np.array([80, 75, 90, 85])
```

Lists are general-purpose containers. NumPy arrays are optimized for numerical operations and multidimensional data.

5. 1D, 2D and 3D Arrays

```
arr1 = np.array([10, 20, 30, 40])

arr2 = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

arr3 = np.array([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
])

print(arr1.ndim)
print(arr2.ndim)
print(arr3.ndim)
```

6. ndim, shape, size and dtype

```
arr = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

print("Dimensions:", arr.ndim)
print("Shape:", arr.shape)
print("Size:", arr.size)
print("Data type:", arr.dtype)
```

ndim = number of dimensions. **shape** = dimensions such as rows and columns. **size** = total elements. **dtype** = element data type.

7. Array Creation Functions

```
print(np.zeros(5))
print(np.ones((2, 3)))
print(np.full((2, 3), 5))
print(np.eye(3))
```

8. arange()

```
print(np.arange(1, 10))
print(np.arange(0, 20, 2))
```

Syntax: np.arange(start, stop, step). The stop value is normally excluded.

9. linspace()

```
arr = np.linspace(0, 10, 5)
print(arr)
```

linspace() creates a chosen number of evenly spaced values between two limits.

10. Indexing

```
marks = np.array([80, 75, 90, 85])

print(marks[0])
print(marks[1])
print(marks[-1])
```

11. 2D Indexing

```
marks = np.array([
    [80, 75, 90],
    [70, 85, 95]
])

print(marks[0, 0])
print(marks[0, 1])
print(marks[1, 2])
```

Use array[row, column]. Indexing starts from 0.

12. Changing Values

```
marks = np.array([80, 75, 90, 85])
marks[1] = 78
print(marks)
```

13. Slicing

```
marks = np.array([80, 75, 90, 85, 95])

print(marks[1:4])
print(marks[:3])
print(marks[2:])
```

The start index is included and the stop index is excluded.

14. Array Arithmetic

```
marks = np.array([80, 75, 90, 85])

print(marks + 5)
print(marks * 2)
print(marks / 2)
```

15. Array + Array

```
midterm = np.array([40, 35, 45, 38])
```

```
final = np.array([45, 40, 42, 44])

total = midterm + final
print(total)
```

16. Statistical Functions

```
marks = np.array([80, 75, 90, 85])

print("Sum:", np.sum(marks))
print("Mean:", np.mean(marks))
print("Maximum:", np.max(marks))
print("Minimum:", np.min(marks))
print("Standard Deviation:", np.std(marks))
```

17. reshape()

```
arr = np.arange(1, 13)
matrix = arr.reshape(3, 4)
print(matrix)
```

The total number of elements must remain unchanged. Twelve elements can become 3×4 , but not 3×5 .

18. Random Numbers

```
numbers = np.random.randint(1, 101, 10)
print(numbers)

decimals = np.random.rand(5)
print(decimals)
```

19. Boolean Indexing

```
marks = np.array([45, 67, 89, 32, 76, 91, 55])

print(marks[marks >= 50])
print(marks[marks >= 80])
```

Boolean filtering selects values that satisfy a condition. This pattern is extremely important in Data Science.

20. Practical CGPA Analysis

```
cgpa = np.array([3.23, 3.10, 2.90, 3.60, 3.45, 2.75])

print("Average:", np.mean(cgpa))
print("Highest:", np.max(cgpa))
print("Lowest:", np.min(cgpa))
print("CGPA >= 3.0:", cgpa[cgpa >= 3.0])
```

21. Mini Project: Student Marks Analyzer

```
import numpy as np

marks = np.array([
    85, 72, 91, 65, 78,
    88, 95, 69, 55, 81
])

print("Student Marks:", marks)
print("Total Marks:", np.sum(marks))
print("Average Marks:", np.mean(marks))
print("Highest Marks:", np.max(marks))
print("Lowest Marks:", np.min(marks))
print("Standard Deviation:", np.std(marks))

passing = marks >= 50
above_80 = marks > 80

print("Passing Students:", np.sum(passing))
print("Students Above 80:", np.sum(above_80))

pass_percentage = np.mean(passing) * 100
print("Pass Percentage:", pass_percentage)
```

22. Practice Tasks

1. Create a NumPy array containing 10, 20, 30, 40 and 50.
2. Create `[[10, 20, 30], [40, 50, 60]]` and display shape, size and dimensions.
3. Create an array from 1 to 20 using `arange()`.
4. Create 10 evenly spaced values from 0 to 100 using `linspace()`.
5. Create five zeros and a 3×3 matrix of ones.
6. For `[80, 75, 90, 85, 95]`, calculate total, average, maximum, minimum and standard deviation.
7. From `[45, 67, 89, 32, 76, 91, 55]`, select marks ≥ 50 .
8. Create 1 to 12 and reshape it into a 3×4 matrix.
9. For CGPAs `[3.23, 3.10, 2.90, 3.60, 3.45, 2.75]`, calculate average, highest, lowest and values ≥ 3.0 .
10. Build a Student Marks Analyzer showing total, average, highest, lowest, standard deviation, passing students, students above 80 and pass percentage.

23. Complete Solutions

Task 1 Solution

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print(arr)
```

Task 2 Solution

```
import numpy as np
arr = np.array([[10, 20, 30], [40, 50, 60]])
print("Shape:", arr.shape)
print("Size:", arr.size)
print("Dimensions:", arr.ndim)
```

Task 3 Solution

```
import numpy as np
print(np.arange(1, 21))
```

Task 4 Solution

```
import numpy as np
print(np.linspace(0, 100, 10))
```

Task 5 Solution

```
import numpy as np
print(np.zeros(5))
print(np.ones((3, 3)))
```

Task 6 Solution

```
import numpy as np
marks = np.array([80, 75, 90, 85, 95])
print("Total:", np.sum(marks))
print("Average:", np.mean(marks))
print("Maximum:", np.max(marks))
print("Minimum:", np.min(marks))
print("Standard Deviation:", np.std(marks))
```

Task 7 Solution

```
import numpy as np
marks = np.array([45, 67, 89, 32, 76, 91, 55])
print(marks[marks >= 50])
```

Task 8 Solution

```
import numpy as np
arr = np.arange(1, 13)
print(arr.reshape(3, 4))
```

Task 9 Solution

```
import numpy as np
cgpa = np.array([3.23, 3.10, 2.90, 3.60, 3.45, 2.75])
print("Average:", np.mean(cgpa))
print("Highest:", np.max(cgpa))
```

```
print("Lowest:", np.min(cgpa))
print("CGPA >= 3.0:", cgpa[cgpa >= 3.0])
```

Task 10 Solution

```
import numpy as np
marks = np.array([85, 72, 91, 65, 78, 88, 95, 69, 55, 81])
print("Total:", np.sum(marks))
print("Average:", np.mean(marks))
print("Highest:", np.max(marks))
print("Lowest:", np.min(marks))
print("Standard Deviation:", np.std(marks))
passing = marks >= 50
above_80 = marks > 80
print("Passing Students:", np.sum(passing))
print("Students Above 80:", np.sum(above_80))
print("Pass Percentage:", np.mean(passing) * 100)
```

24. Quick Revision

Command / Property	Purpose
np.array()	Create an array
np.zeros()	Create zeros
np.ones()	Create ones
np.full()	Fill with a chosen value
np.eye()	Create identity matrix
np.arange()	Create a numeric sequence
np.linspace()	Create evenly spaced values
array.ndim	Number of dimensions
array.shape	Rows/columns or other dimensions
array.size	Total elements
array.dtype	Element data type
array.reshape()	Change array shape
np.sum()	Sum
np.mean()	Average
np.max()	Maximum
np.min()	Minimum
np.std()	Standard deviation

25. Common Mistakes

- Forgetting import numpy as np.
- Confusing shape with size.
- Forgetting that indexing starts at 0.
- Trying an incompatible reshape.
- Forgetting that arange() normally excludes stop.
- Confusing list operations with NumPy array operations.
- Forgetting Boolean indexing when filtering.

26. Data Science Connection

NumPy is a major bridge from Python programming to Data Science. Arrays, vectorized operations, aggregation, reshaping and Boolean filtering will appear repeatedly in data analysis, statistics and Machine Learning.

27. Final Checklist

- I understand what NumPy is.
- I can create 1D and 2D arrays.
- I understand ndim, shape, size and dtype.
- I can use zeros, ones, full and eye.
- I can use arange and linspace.
- I can index and slice arrays.
- I can perform arithmetic on arrays.
- I can calculate mean, sum, min, max and standard deviation.
- I can reshape an array.
- I can generate random data.
- I can filter arrays with Boolean conditions.
- I can build the Student Marks Analyzer.

Study rule: Type the examples yourself, complete Tasks 1–10 without copying first, then compare with the solutions. The goal is understanding and coding ability, not memorization.